

Gestural-Sound Communication Glove for deaf-mute people based on neural networks and IOT

Jordan Delgado Gutierrez
Faculty of Mechanical Engineering
National University of Engineering
Lima, Perú
jordan.delgado.g@uni.pe

Abstract—This paper presents the design and implementation of a sign language translation glove that utilizes a sequential neural network for gesture prediction. The glove incorporates flex sensors to measure finger voltages and an accelerometer/gyroscope for orientation detection. It also includes a multiplexer, a 3.7V 450mAh battery with a 4056 charging module, and a flat micro vibration motor. The sensor inputs are processed using MicroPython on a Raspberry Pi Pico W, and the collected samples are used for training the neural network. The glove communicates with a web page using the MQTT protocol and Node-RED. The web page, developed using HTML, CSS, and JavaScript, displays the translated sign language gestures. The paper provides an overview of the problem, describes the methodology employed, presents the results obtained, and concludes with the key findings and future directions for improvement.

Index Terms—Sign language translation glove, sequential neural network, flex sensors, accelerometer/gyroscope, MQTT protocol, Node-RED, MicroPython, Raspberry Pi Pico W

I. INTRODUCTION

Sign language serves as a crucial mode of communication for individuals with hearing impairments. However, the language barrier between sign language users and non-sign language users can hinder effective communication. Peruvian Sign Language (LSP) is the recognized sign language used by the Deaf community in Peru. In 2018, it gained official recognition as a national language, a significant step toward promoting inclusion and protecting the rights of Deaf individuals. Sign language interpreters are in high demand, facilitating communication between Deaf and hearing individuals across various settings. Efforts have been made to integrate sign language into the education system, ensuring equal access to education for Deaf students. Dedicated Deaf organizations tirelessly advocate for the rights and well-being of the Deaf community in Peru. It's worth noting that regional variations or dialects of LSP may exist, similar to spoken languages with regional accents or dialects. The media has also embraced the importance of sign language, incorporating interpretation services during television broadcasts and news programs. According to the National Population Census (INEI-2017), there are 232 176 people with hearing impairment in Peru, a figure that represents 7.6 % of people with disabilities [1]. According to this census, 8790 people with disabilities registered Peruvian Sign Language (LSP) as their mother tongue. Although

there is no precise information on the number of people who currently communicate through PSL, the Ombudsman's Office constantly receives complaints and petitions referring to the barriers faced by persons with disabilities in educational and health services, among others, due to the lack of qualified teachers and interpreters in PSL. In the educational field, the Ombudsman's Office warned, in 2019, that 76% of public schools and 83% of private schools acknowledge that they are not able to provide educational services to deaf students. Likewise, the exclusion of deaf students at the secondary level was alerted, since the 2019 School Census reported 587 deaf students at the primary level and 350 at the secondary level. To address this problem, several projects and technologies, such as sign interpreter gloves, have been developed to improve communication between deaf and dumb people and those who do not speak sign language. Previous precedents have attempted to address sign language translation, but have faced difficulties in data collection and component integration. These projects have employed conventional and limited approaches in terms of accuracy and efficiency:

- **SignAloud:** Developed by two students at the University of Washington, SignAloud was a glove that used flex sensors to capture hand and finger movements while performing American Sign Language (ASL). These movements are then translated into text or speech.
- **University of the Armed Forces (Lacatunga, Ecuador):** This sign translator glove project also utilizes flex sensors and an accelerometer, in addition to an Arduino Nano Atmega328 controller and Bluetooth communication with mobile devices for the visualization of the signs.
- **Escuela Politecnica Nacional (Quito, Ecuador):** As in the other two cases, flex sensors were used. For IoT communication we used an ESP6682 with a Wifi module, in addition to an Arduino.
- **Instituto Politecnico Nacional (Azcatpotzalco, Mexico):** This project is able to recognize the Mexican alphabet as well as some abbreviations that make up the Mexican sign language. It used flex sensors, accelerometer for data capture. For text display an LCD screen and an Emic 2 module to convert text to speech. It also applied digital signal conditioning and a set of conditionals for sign

detection.



Fig. 1. There are more than 232 thousand deaf and hard-of-hearing individuals in Peru [1]

II. PROBLEM STATEMENT

The lack of effective communication between individuals who use sign language and those who do not poses significant challenges in various social and professional settings. Traditional sign language translation methods rely on human interpreters, which may not always be available or practical. Automated sign language translation systems offer a solution to overcome these challenges. However, existing systems often have limitations in terms of accuracy, speed, and accessibility. Therefore, there is a need for an innovative and efficient sign language translator that can overcome these limitations and facilitate seamless communication between sign language users and non-sign language users.

III. METHODOLOGY

The glove incorporates flex sensors to measure finger tension and an accelerometer/gyroscope to detect orientation. It also includes a multiplexer, a 3.7 V 450 mAh battery with a 4056 charging module, and a flat vibrating micromotor. Sensor inputs are processed with MicroPython on a Raspberry Pi Pico W, and the collected samples are used to train the neural network. The glove communicates with a web page using the MQTT and Node-RED protocol. Finally, the web page, developed with HTML, CSS and JavaScript, shows the gestures translated from sign language. The block diagram of the project is shown in Figure 2

A. Hardware Design and Implementation

1) *Raspberry Pi Pico W*: For the selection of the microcontroller, the following factors were taken into consideration: availability, price, wireless connectivity, processing power, wide range of GPIO pins, and versatile programming. Meeting all these characteristics, the Pico W was chosen.

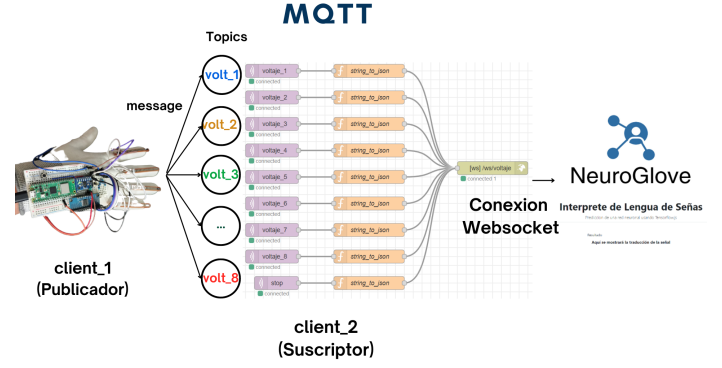


Fig. 2. Project block diagram

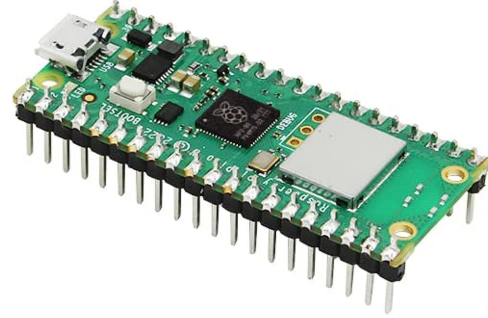


Fig. 3. Raspberry Pi Pico W

2) *Sensor Flex*: To detect finger movement information, it was decided to use flex sensors, which are variable resistances that change with flexion. Although there are alternative options such as resistive force sensors, optical flex sensors, or magnetic flex sensors, flex sensors were chosen because they were the only ones available in the market and also the most cost-effective.

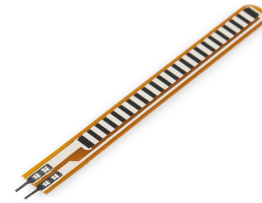


Fig. 4. Sensor flex

3) *Multiplexor*: Due to the limited number of ADC (Analog-to-Digital Converter) pins on the Pico Pi W, which is only 3, and the requirement of 5 ADC inputs for each finger, it was decided to use a 16-channel multiplexer, specifically the 74HC4067. This allows us to multiplex the signals so that only one ADC pin of the Pico W is used at a time for the signal of one of the 5 fingers.

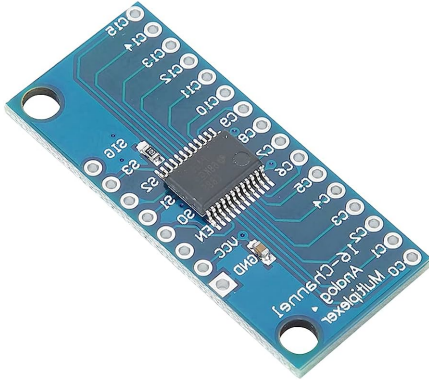


Fig. 5. Multiplexor 74HC4067

4) *Accelerometer*: Since some gestures involve not only finger movement but also the position of the entire hand, it was decided to implement an accelerometer/gyroscope to obtain 3 additional data points (linear acceleration in each axis). This allows us to enrich the range of samples and capture more comprehensive information about hand motion.

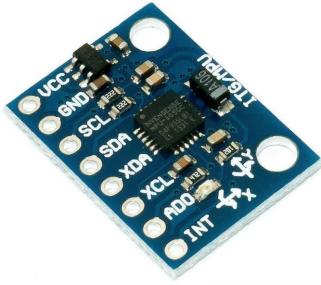


Fig. 6. Accelerometer MPU6050

5) *Vibratory micro motor module*: Initially, the blinking of an LED was used as a warning signal before taking a new sample. However, since the glove is in motion most of the time, continuously checking the LED on the Pico W became impractical. Therefore, it was decided to incorporate a small vibration module as a complementary warning signal alongside the LED. This allows for a more convenient and noticeable indication during the sampling process.

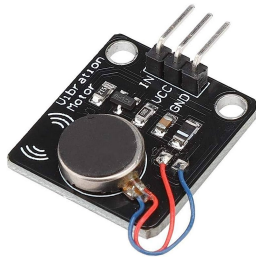


Fig. 7. Vibratory micro motor module

6) *Feeding*: To make the glove portable, a small-sized 3.7V, 450mAh lithium battery was chosen. It is attached to the glove along with the previous actuator. Additionally, a small charging module was implemented to facilitate the charging process of the battery. These components together enable the glove to be self-contained and easily rechargeable.



Fig. 8. Referential Image

7) *Final Design*: The circuit implementation is shown in Figure 2.5. For this, an antistatic glove was used, 2 mini breadboards of 25 points each in which the resistors for the voltage divider were placed, and 1 breadboard of 400 points to place the raspberry pi pico w, the MPU6050 accelerometer, a multiplexer (to use 5 ADC) and a push button. In addition, a micro vibration motor module and a 3.7 V 450 mAh Lithium battery with a TP4056 charging module were implemented to make the device portable. Double-sided tape was used to adhere the breadboards and a part of the flex sensors to the glove. Finally, to join the flex sensors to the breadboard, female jumpers were soldered to each of the pins. The final system implemented can be seen in the following figure.

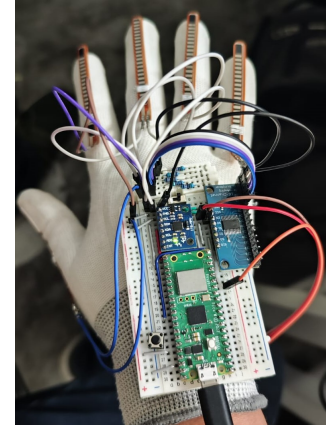


Fig. 9. Referential Image

B. Data collection

For data collection, 5 flex sensors were used, one for each finger of the right hand, each of which acts as a potentiometer that varies its resistance according to its degree of flexion. What is done in the implementation is to make a voltage divider with the flex sensors, which returns an analog voltage value. To read 5 analog voltage values with the raspberry pi pico w, a multiplexer was used, since the pico w It only has 3 ADCs. In the micropython code (python language adapted for microcontrollers) said voltages of each flex sensor were stored in 5 variables (voltage1-voltage5), in addition to that the ax, ay and az parameters provided by the MPU6050 accelerometer

were used, this due to that it was noticed that in the sign language alphabet there are some signs that are very similar to others, so the only way for the neural network to distinguish these signs is from 3 additional parameters that represent the position on the x axis, and and z. In total, 8 variables were collected, which were compiled in a table of 9 columns and 13,500 rows: The first column represents the target, which is the token of the letter that represents said combination of voltages (from 10 to 36), the other 8 columns are the voltage values for each finger, and the 13,500 rows represent 500 samples for each letter of the alphabet (27 times 500). A portion of the table is shown in Figure 2.1.

	y	dedo_1	dedo_2	dedo_3	dedo_4	deo_5	ax	ay	az
0	10	1.755972	1.697963	1.753555	2.022701	0.035450	-0.891357	0.167725	-0.221191
1	10	1.747109	1.689906	1.754360	2.004976	0.026587	-0.926758	0.224609	-0.284180
2	10	1.747109	1.690712	1.750332	2.003365	0.033033	-0.910889	0.183838	-0.267090
3	10	1.743886	1.693934	1.748721	2.007393	0.036255	-0.917969	0.170166	-0.246826
4	10	1.753555	1.735024	1.755166	1.998530	0.026587	-0.924561	0.172363	-0.263916
5	10	1.741469	1.711659	1.739858	2.005782	0.027393	-0.910400	0.197998	-0.258057
6	10	1.747915	1.692323	1.731801	1.999336	0.030616	-0.911133	0.187988	-0.253174
7	10	1.754360	1.701991	1.757583	2.007393	0.023365	-0.905273	0.188721	-0.260010
8	10	1.753555	1.711659	1.775308	2.013033	0.023365	-0.919434	0.187988	-0.265625
9	10	1.749526	1.701186	1.755166	2.009810	0.039478	-0.913574	0.199219	-0.262451
10	10	1.748721	1.696352	1.723744	1.987251	0.038672	-0.912842	0.190918	-0.268066
11	10	1.751943	1.702797	1.727773	2.001753	0.020142	-0.924072	0.188477	-0.258057
12	10	1.760000	1.698769	1.774502	2.030758	0.029004	-0.924805	0.187500	-0.248047
13	10	1.743081	1.710854	1.750332	2.009004	0.038672	-0.914551	0.187744	-0.256592
14	10	1.747109	1.710854	1.755166	2.001753	0.013697	-0.914307	0.186523	-0.257080
15	10	1.747109	1.724550	1.731801	2.003365	0.020948	-0.911865	0.194336	-0.253906
16	10	1.750332	1.760000	1.768862	2.003365	0.040284	-0.898682	0.188965	-0.257568
17	10	1.749526	1.719716	1.747915	2.009810	0.035450	-0.915527	0.180420	-0.257324
18	10	1.743081	1.734218	1.754360	2.040426	0.022559	-0.924805	0.191894	-0.260742

Fig. 10. Network training dataset

A behavioral analysis of the data was also performed. The behavior of voltage 1 was analyzed through each row of the dataset and it can be seen how 500 in 500 behaves in an approximately constant manner, this is because the same letter is being sampled and it corresponds correctly to have said behavior. The same analysis was performed with each letter (Figure 2.2).

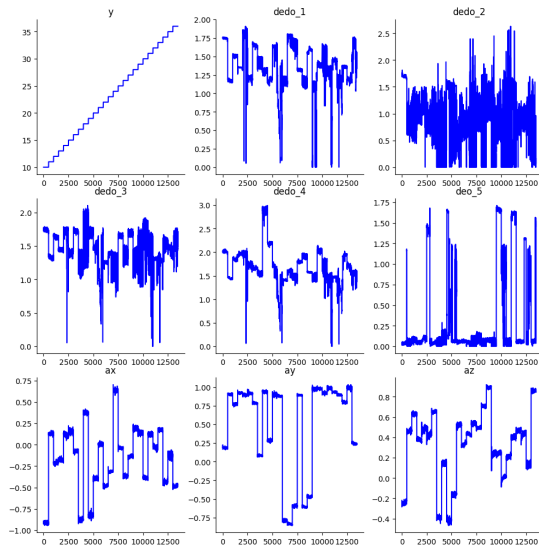


Fig. 11. voltage behavior analysis 1

C. Creation and training of the neural network

As can be seen in Figure 2.3, 1 input layer and 8 hidden layers have been used, in each of which the "relu" activation function was used, which allows solving a variety of much more complex problems, finally for the output layer. The linear activation function was used because the outputs need to be numbers from 10 to 36, this is because each letter of the alphabet was tokenized with a symbolic number, for example the number 10 would represent the letter A, 11 a the letter B and so on, this is because the neural network works with numbers and weights.

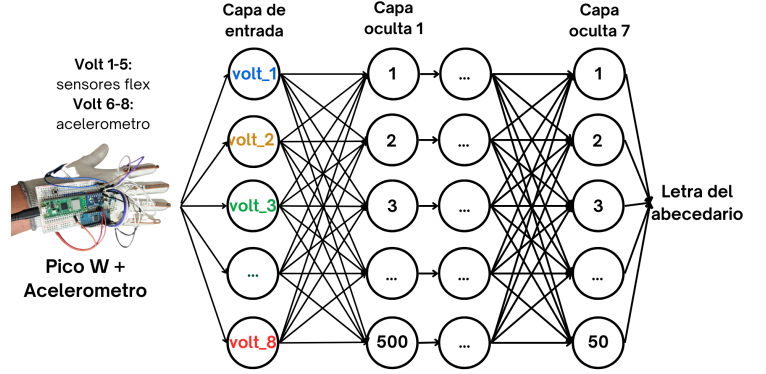


Fig. 12. Neural Network of 8 hidden layers

In Figure 2.4 you can see the parameters of the neural network: A total of 9 layers were used. In the input layer there are 8 neurons. Hidden layer 1 and 2 use 500 neurons each and hidden layer 3 to 5 use 200 neurons each layer. Hidden layer 6 uses 100 neurons, hidden layer 7 uses 50 neurons, and finally hidden layer 8 uses 1 neuron. The parameters of each layer (the weights) are shown in the table.

	Layer Name	Input Shape	Output Shape	Activation	Number of Parameters
0	dense	(None, 8)	(None, 500)	relu	4500
1	dense_1	(None, 500)	(None, 500)	relu	250500
2	dense_2	(None, 500)	(None, 500)	relu	250500
3	dense_3	(None, 500)	(None, 200)	relu	100200
4	dense_4	(None, 200)	(None, 200)	relu	40200
5	dense_5	(None, 200)	(None, 200)	relu	40200
6	dense_6	(None, 200)	(None, 100)	relu	20100
7	dense_7	(None, 100)	(None, 50)	relu	5050
8	dense_8	(None, 50)	(None, 1)	linear	51

Fig. 13. table of parameters

D. Creation of the MQTT broker in MyQTTHub

A mqtt broker with a free plan was created on the MyQTT-THub page, this broker was chosen due to its friendly and intuitive interface, in addition to having a free plan with many important features such as the creation of multiple clients and a capacity of 8000 messages per hour. The following figure shows the tests carried out publishing messages from the Raspberry Pi Pico W.

Stashed messages at PublishedMessages, for domain hub-JordanDelgado

Refresh Last 24 hours Download Stash Manage Stash Clear Stash

Stashed messages 20

Clientid	User name	Qos	Retain	Dup	Remote address / Date	Topic / Payload
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:42 CEST 2023	voltaje_8 0.8569336
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:42 CEST 2023	voltaje_7 -0.2062988
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:42 CEST 2023	voltaje_6 -0.472168
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:42 CEST 2023	voltaje_5 0.04189517
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:42 CEST 2023	voltaje_4 1.65929
gestos_client	jordandelgado	0	-	-	179.6.72.201 Tue Jul 11 04:31:41 CEST 2023	voltaje_3 1.486825

Fig. 14. Test to publish messages from Pico W

E. Raspberry Pi Pico W connection and MQTT broker

For the connection of the Raspberry Pi Pico W the IDE Thonny was used. To start, 2 important libraries are imported for the connection: the first is the "network" library for the wifi connection of the pico w and the other important library is umqtt.simple for the connection with the mqtt broker.

```
import machine

from imu import MPU6050 #necesitamos libreria imu y vector3d

import network

import utime

from umqtt.simple import MQTTClient
```

Fig. 15. Librerías a usar

First, it connects to the internet by placing the respective credentials with the following code.

```
ssid = "Neuroglove network"
password = "mypassword"

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.config(pm = 0xa11140)
wlan.connect(ssid, password)
```

Fig. 16. Conexión wifi

Then the mqtt connection is made with the broker previously created in myqttHub. For this, the MQTTClient function is used, to which the broker parameters such as url, username, password and clientid must be passed. This is shown in Figure 2.9.

```
def connectMQTT():
    client = MQTTClient(client_id="gestos_client",
                        server="node02.myqttHub.com",
                        port=0,
                        user="jordandelgado",
                        password="bryanysebas",
                        keepalive=7200,
                        ssl=True,
                        ssl_params={'server_hostname': 'node02.myqttHub.com'})
    print("Conectando al broker")
    client.connect()
    print("Broker conectado")
    return client

client = connectMQTT()
```

Fig. 17. MQTT Conection

F. Node-RED and MQTT broker connection

For the connection of the MQTT broker and Node-Red, the "mqtt in" nodes are used, which allow subscribing to the desired topic to receive the messages published by Pico W. In total, 8 nodes are used (1 for each voltage), in each one the broker's credentials had to be entered, which allowed the successful connection with it. This can be seen in the green box that says "connected" in Figure 2.10. In addition to this, the "function" node is used, whose function is to convert from string to JSON, this is due to the fact that the messages published by Pico W are character strings, so this function allows reading these values without problems to later send them to the "websocket" node that is the fundamental piece to connect the broker with the web page using node-red as an intermediary. In this node we place "ws/voltage" which is an identifier that allows the connection to the web page. All this connection can be seen in Figure 2.10.

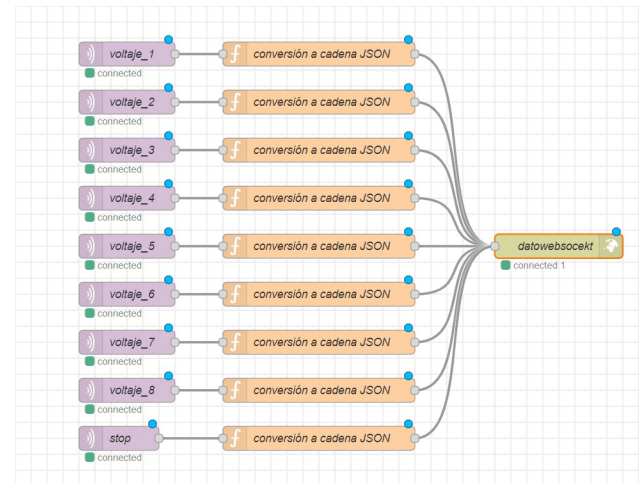


Fig. 18. Flow Diagram Node-Red

G. Node-RED connection and web page in HTML and JavaScript via WebSocket

A Javascript script containing the code for the websocket connection to node-red is placed in the HTML of the web page: a websocket connection through node-red is used due

to its ease of use and because the broker does not have a built-in websocket feature so you can't connect directly to the web page. Notice in Figure 2.11 that the identifier "ws/voltage" was used.

```
// Reemplaza la URL con la dirección de tu instancia de Node-RED
const socket = new WebSocket('ws://127.0.0.1:1880/ws/voltaje');

// Abre la conexión
socket.onopen = function(event) {
  console.log('Conexión websocket abierta');
};

// Escucha los mensajes entrantes
socket.onmessage = function(event) {
  const mensaje = JSON.parse(event.data);
  const topico = mensaje.topico;
  const valor = mensaje.valor;
};
```

Fig. 19. Conexión websocket

H. Conexión página web y ngrok

Por último, para poder utilizar la página web desde cualquier dispositivo, esto incluyendo celulares de cualquier tipo, se emplea ngrok que permite generar un dominio gratuito y poder tener la página web en la red. Cada vez que se reinicia ngrok, se genera una nueva URL aleatoria para la aplicación.

IV. RESULTS

It can be seen on Figure 3.1 how the neural network lost decrease considerably around the 600th epochs, which means that the network improved in each epoch the best combination of weights possible from the network that allows you to obtain very accurate results. Note also that the blue curve represents the error obtained with the training data (70% of the dataset) and the orange curve is the error obtained with the test data (30% of the dataset) The Figure shows a comparison of the

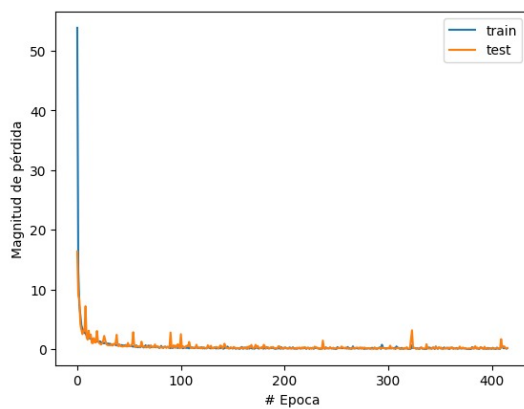


Fig. 20. Loss magnitude versus Epochs

predictions of our network with the test data (expected data), comparing these results an error of 0.8

	y	y_predict
428	10	10.0
1059	14	14.0
2893	25	25.0
1869	18	18.0
1863	18	18.0
4092	31	31.0
340	10	10.0
4334	31	31.0
5194	17	17.0
1511	14	14.0
5932	21	21.0
839	10	10.0
1681	14	14.0
4171	31	31.0
4924	17	17.0
6283	21	21.0
3952	31	31.0
4792	17	17.0
2992	25	25.0

Fig. 21. Comparison of network predictions and expected value

Finally, the Figure shows a real test of the glove in action, publishing messages to the broker and this at the same time sending it to the web page through a websocket connection in node-red, on the right side of the screen you can see the console of the web page where the published voltages are displayed. and how it is instantly updated on the web page, after that the network receives those voltages as input and performs the prediction of the letter, said letter will be concatenated with the next letter sent after 2 and a half seconds (parameter that can be changed to comfort), these letters will be concatenated until the user decides to stop the sending by means of a button, which when pressed publishes a message "-1" in the "stop" topic that, when it arrives at the web page, detects it as a signal to stop predicting data and to start showing the result on the web page, in addition to reproducing the concatenated text with the voice of Google in Spanish.

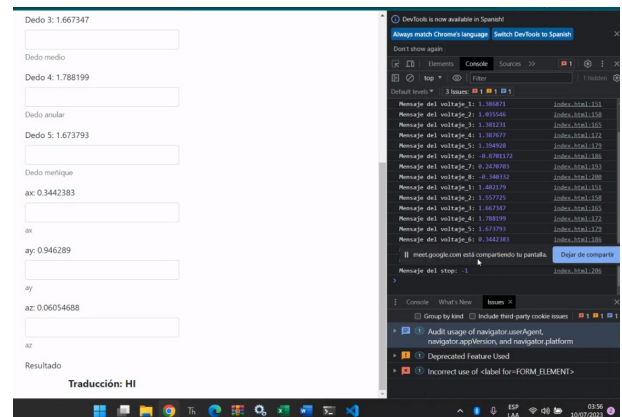


Fig. 22. Web page and Console receiving data

V. CONCLUSIONS AND RECOMMENDATIONS

A. Conclusions

- A sign translator glove was successfully implemented within the planned scope.

- A neural network with an error of 0.8 % (total error 16, assuming error of 1 in each wrong sample) was created in a universe of 1890 new samples.
- It was possible to create a web page with a friendly and easy-to-understand dashboard, where the voltage values and the final prediction are shown.
- It was possible to implement, by means of an algorithm on the web page, a system for recognizing letters and words shown as text and also reproducible through audio.
- Virtually instantaneous communication with the web page was achieved, allowing rapid delivery of data and translations.

B. Recommendations

- Due to the nature and size of the breadboard, the double-sided tape does not cover the entire surface of the base, so it does not remain firmly attached to the glove. It is recommended to design a PCB of the appropriate size that can be properly attached to the glove, in addition to significantly reducing the use of jumpers, making the prototype more ergonomic.
- Despite having an error of 0.8%, the network still needs a much larger sample size in training since errors are still detected.

REFERENCES

- [1] Retrieved from <https://www.fayerwayer.com/ciencia/2022/09/07/jovenes-del-mit-desarrollan-guantes-para-que-personas-sordomudas-puedan-hablar-pero-todo-sale-mal/>
- [2] Almeida Pozo, L., Viteri Villacís, P. (2016). Diseño e implementación de un par de guantes intérpretes del lenguaje de señas elementales a lenguaje escrito mediante software libre para facilitar el aprendizaje en la Unidad educativa especializada cotopaxi. Para la obtención del título de Ingeniería. Departamento deficiencias de la energía y mecánica de la Universidad de las Fuerzas Armadas.
- [3] Males Maldonado, E. D., Crespo Crespo, K. C. (2022). Guante electrónico traductor de señas a través de comunicación wifi (Bachelor's thesis, Quito, 2022).